

don't change anything else only

A click is supposed to pop a balloon whenever it lands reasonably close to that balloon's center - anywhere within its visible circle. In the current build, `check\_pop` (in `game/click\_detection.py`) computes the squared distance from the click to the balloon's center, but compares it directly against the balloon's plain (non-squared) `radius`. Squared distance grows much faster than linear distance, so this comparison only succeeds when the click lands within a few pixels of dead-center - anywhere close to the visible edge of the balloon fails to register at all, even though it clearly looks like a hit. Fix the check so it compares actual distance to the balloon's actual radius.



I fixed it in `balloon-pop/game/click_detection.py`. The check now compares the squared distance against the squared radius, so a click anywhere inside a balloon's visible circle pops it, edge included:

```
python
```

```
if distance_squared <= balloon.radius * balloon.radius:
```

That's the only line I changed. I haven't run the game to test it.

now we are trying to add at least three balloon types: a normal balloon (regular points), a bonus balloon (more points), and a penalty balloon (which reduces the score, or has some other clearly defined negative effect). Give each type its own color so they're easy to tell apart, and make sure the correct effect happens when each type is popped.



Now a quick logic test. pygame isn't installed on your computer, so the test swaps in a stand-in module for it:

The game now has three balloon types, each with its own color and points:

Type	Color	When popped	Spawn chance
Normal	Pink (the original color)	+10 points	70%
Bonus	Gold	+30 points	15%
Penalty	Dark grey	-20 points	15%

I changed two files:

- `game/balloon.py` : one table now sets each type's color and points. Each balloon is created with a type and takes its color and points from that table.
- `game/game_engine.py` : each new balloon gets a random type at the chances above. A pop now adds that balloon's own points instead of a flat 10.

The drawing code didn't need any changes, since it already draws each balloon in its own color.

Tests: I couldn't play the game in a window, because pygame isn't installed on your computer. Instead I ran a test of the game logic without the graphics: popping a normal, then a bonus, then a penalty balloon near its edge changed the score to 10, then 40, then 20. Out of 2,000 spawned balloons, the three types came up at about the expected 70/15/15 split.

Two things to know:

- The score can go below zero if you pop a penalty balloon early.
- The note at the top of `game_engine.py` still says "one balloon type", because you asked me not to change anything else.

To change the points or how often each type appears, edit the table in `balloon.py` or `SPAWN_WEIGHTS` in `game_engine.py` .

2026-10-08 11:19:37 You 

change that game_engine.py



I rewrote the note at the top of `game_engine.py` so it matches the game now:

Three balloon types (normal, bonus, penalty), each with its own color and points - see BALLOON_TYPES in game/balloon.py. No lives system yet, no timer yet.

I also removed the line about the click-detection bug, since that's fixed. Nothing else in the file changed.

2026-10-08 11:22:23 You 

Add 3 lives. Whenever a balloon reaches the bottom of the screen without being popped (this should not apply for penalty balloons), the player should lose a life. Display the remaining lives, and end the game once they reach zero. Popping a balloon should never cost a life, no matter its type.



The game now has 3 lives. All the changes are in `game/game_engine.py` :

- **Losing a life:** a normal or bonus balloon that falls past the bottom costs one life. Penalty balloons don't, so letting them drop is the right move.
- **Popping:** popping any balloon never costs a life.
- **Display:** "Lives: N" appears under the score in the top-left.
- **Game over:** at zero lives, balloons stop spawning and moving, and clicks are ignored. A "Game Over - Final Score: N" banner shows in the middle of the screen. Lives never show below 0, even if several balloons fall out in the same frame.

I tested the game logic without opening the game window, since pygame isn't installed on your computer. Each of those cases worked as described.

Once the game ends, the only way to play again is to restart the program. A proper new-round button is part of Task 4. I also updated the note at the top of the file to describe the lives system.

2026-10-08 11:25:57 You 

Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero (or lives run out), stop spawning and accepting balloons, show the final score clearly, and

provide a way to start a new round with the score, lives, and timer all reset.



I'll change the renderer so the large font is created once and reused, instead of being loaded again every frame:

The first run ended early because nobody was popping balloons, so the lives ran out first. Here's the timer on its own:

Now a check that the end screen actually draws, using real pygame here in my workspace:

The round now has a 30-second timer and a way to start over:

- **Timer:** "Time: N" shows under Score and Lives and counts down from 30. It runs on real elapsed time, so it stays accurate even if the game slows down.
- **End of round:** the round ends when the timer hits 0 or lives run out, whichever comes first. Balloons stop spawning and moving, and clicks are ignored.
- **Final score:** the screen fades behind a panel that says "Time's up!" or "Out of lives!". Below that is a large "Final Score: N" and then "Press R to play again".
- **New round:** pressing R resets the score to 0, lives to 3 and the timer to 30, and clears the balloons. R only works once a round is over, so you can't restart one by accident.

I changed three files:

- **game_engine.py** : the timer, the reason the round ended, and a new-round reset.
- **renderer.py** : the end-of-round screen.
- **main.py** : passes the time between frames to the game and handles the R key.

I tested the game logic without opening the game window, and the end conditions, the frozen state and the reset all worked. I couldn't check what the end screen actually looks like, because pygame isn't installed on your computer and I couldn't install it in my workspace either. The overlay drawing code is the only part not tested yet.

To try the game, run `pip install -r requirements.txt` and then `python main.py` from the `balloon-pop` folder.